

 Roteiro de Apresentação Rápida — Frontend AutoMatch

Este guia foi desenhado para uma **apresentação dinâmica, técnica e direta ao ponto** (cerca de **5 a 7 minutos**). A linguagem foi adaptada para soar natural, profissional e sem formalidades excessivas.

 Minuto a Minuto (Pitch de 6 Minutos)

Tempo	Etapa	Foco Principal
0:00 - 1:00	1. Introdução	O problema e a solução multiplataforma (Web/Mobile)
1:00 - 2:30	2. Arquitetura & Reuso	Estrutura de packages compartilhados, Core e API client
2:30 - 4:00	3. Monorepo & PNPM	Por que usamos pnpm e Turborepo na prática
4:00 - 5:30	4. Live Demo	Telas funcionando, consistência de UI/UX e IHC
5:30 - 6:00	5. Fechamento	Práticas de engenharia e encerramento

 Roteiro de Fala (O que dizer em cada slide)

Slide 1: Introdução — O Ecossistema AutoMatch

• Fala sugerida:

"Fala pessoal, beleza? Hoje vamos direto ao ponto mostrar como estruturamos o frontend do AutoMatch.

O nosso desafio aqui foi criar duas pontas que conversam muito bem: de um lado, o aplicativo **Mobile** (usando Expo) focado no cliente que precisa agendar serviços rápido pelo celular; e do outro, a plataforma **Web** (em Next.js) para o mecânico gerenciar o catálogo e as solicitações. Para não reescrever código e manter tudo alinhado, estruturamos o frontend usando um Monorepo."

Slide 2: A Arquitetura e a Regra de Ouro do Reuso

• Fala sugerida (Mostrando o diagrama de pastas):

"Em vez de criar repositórios separados que duplicam chamadas de API e validações, a gente separou o projeto em duas pastas de aplicações (apps) e duas pastas de pacotes compartilhados (packages):

- **@automatch/core**: Aqui fica o coração das regras de validação usando **Zod** e tipagem estrita do TypeScript. A validação de e-mail e senha do Login, por exemplo, é escrita aqui uma vez só e consumida tanto pela Web quanto pelo Mobile.

- o **@automatch/api-client**: Centraliza nosso cliente **Axios**, interceptores para injetar o token JWT de segurança, o gerenciamento de estados com **Zustand** e o cache local.

Com isso, as nossas aplicações (Web e Mobile) ficam leves, cuidando apenas de renderizar a interface gráfica."

Slide 3: Decisão de Stack — Por que PNPM e Turborepo?

- **Fala sugerida:**

"Para gerenciar tudo isso, a gente escolheu o **pnpm** como gerenciador de pacotes e o **Turborepo** para a orquestração do monorepo. Os motivos foram bem práticos:

1. **Economia de espaço e velocidade (pnpm)**: O pnpm usa uma *Content-Addressable Store* global. Se a Web e o Mobile usam o mesmo pacote, o pnpm baixa apenas uma vez no computador e faz um **Hard Link** físico no disco, tornando instalações quase instantâneas.
2. **Sem dependências fantasmas**: O pnpm usa **Symlinks** (links simbólicos). Isso garante que o código só consiga importar o que está explicitamente declarado no package.json, evitando quebras misteriosas em produção.
3. **Build inteligente (Turborepo)**: O Turborepo roda as tarefas em paralelo usando cache local. Se a gente alterar só o código do Mobile, o build da Web é pulado porque o cache está quente."

Slide 4: Live Demo (Demonstração Prática)

- **Fala sugerida (Mostrando o emulador do celular e o navegador lado a lado):**

"Vamos ver funcionando na prática:

1. **No Mobile (Cliente)**: Ao tentar logar, as validações do Zod rodam localmente no app. Quando clicamos em entrar, o botão de login fica bloqueado e mostra um indicador de carregamento (spinner) para impedir cliques duplos que enviariam dados repetidos para o servidor.
2. Ao solicitar uma revisão no catálogo, o app dispara a requisição real para o Gateway na VPS e exibe um **Toast** escuro e elegante no rodapé (fácil alcance do polegar) confirmando o sucesso.
3. **Na Web (Mecânico)**: O painel Next.js trabalha com **rotas limpas em português** (como **/solicitacoes** e **/configuracoes**). Se redimensionarmos a tela do navegador, a barra lateral vira um menu hambúrguer com Drawer deslizante.
4. O agendamento criado no mobile já aparece instantaneamente na Web. Se o mecânico aprovar, o status é atualizado via PATCH no banco e o cliente vê a mudança em tempo real."

Slide 5: Fechamento & Boas Práticas de IHC

- **Fala sugerida:**

"Para fechar, aplicamos conceitos sólidos de **IHC (Interação Humano-Computador)**:

- **Consistência visual:** Usamos o mesmo design system escuro (Dark Mode por padrão) em todas as telas de ambas as plataformas.
- **Prevenção de erros:** Bloqueios de clique múltiplo em carregamento e validações antes do envio para a API.
- **Feedback imediato:** Toasts customizados e spinners em todas as ações de rede.

Isso nos deu um frontend extremamente rápido de desenvolver, seguro e muito profissional. Valeu galera, obrigado!"